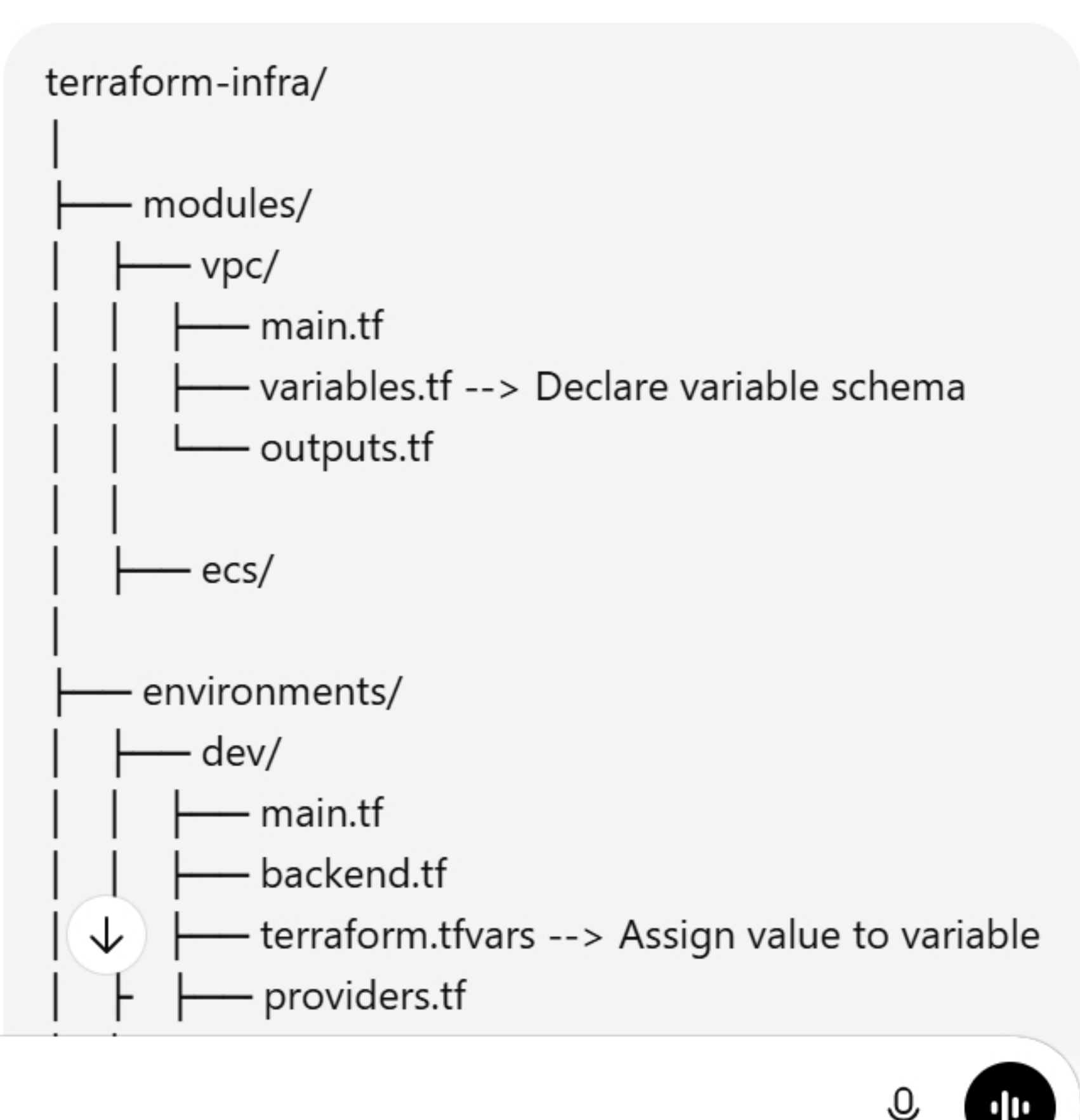




Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

In your Terraform directory structure, the VPC module is reusable infrastructure code, and the dev environment calls that module and passes values to it.

Let's walk through how the `dev` environment uses the `vpc` module.

1 VPC Module (Reusable Code)

Path:

modules/vpc/

This module contains the **actual AWS resources**.

modules/vpc/main.tf

```
hcl
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr

  tags = {
    Name = var.vpc_name
  }
}
```

modules/vpc/variables.tf

Here we **define variables** (schema).

```
hcl
variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type        = string
}

variable "vpc_name" {
  description = "Name of the VPC"
  type        = string
}
```

modules/vpc/outputs.tf

Expose outputs so other modules can use them.

```
hcl
output "vpc_id" {
  value = aws_vpc.main.id
}
```

2 Dev Environment Calls the Module

Path:

environments/dev/main.tf

Here we **call the module**.

environments/dev/main.tf

```
hcl
module "vpc" {

  source = "../../modules/vpc"

  vpc_cidr = var.vpc_cidr
  vpc_name = var.vpc_name
}
```

Explanation:

- `source` → tells Terraform where the module exists
- Variables are passed to the module

3 Dev Variables Definition

Path:

environments/dev/providers.tf

```
hcl
provider "aws" {
  region = "ap-south-1"
}
```

environments/dev/variables.tf (optional but recommended)

```
hcl
variable "vpc_cidr" {
  type = string
}

variable "vpc_name" {
  type = string
}
```

4 Assign Values in terraform.tfvars

Path:

environments/dev/terraform.tfvars

This is where **actual environment-specific values** go.

```
hcl
vpc_cidr = "10.0.0.0/16"
vpc_name = "dev-vpc"
```

5 Backend Configuration

environments/dev/backend.tf

```
hcl
terraform {
  backend "s3" {
    bucket = "terraform-state-dev"
    key    = "vpc/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

This keeps **state separate per environment**.

6 Execution Flow

When you run:

```
cd environments/dev
terraform init
terraform plan
terraform apply
```

Terraform will:

- Load `dev/main.tf`
- See **module "vpc"**
- Go to

../../modules/vpc

Read:

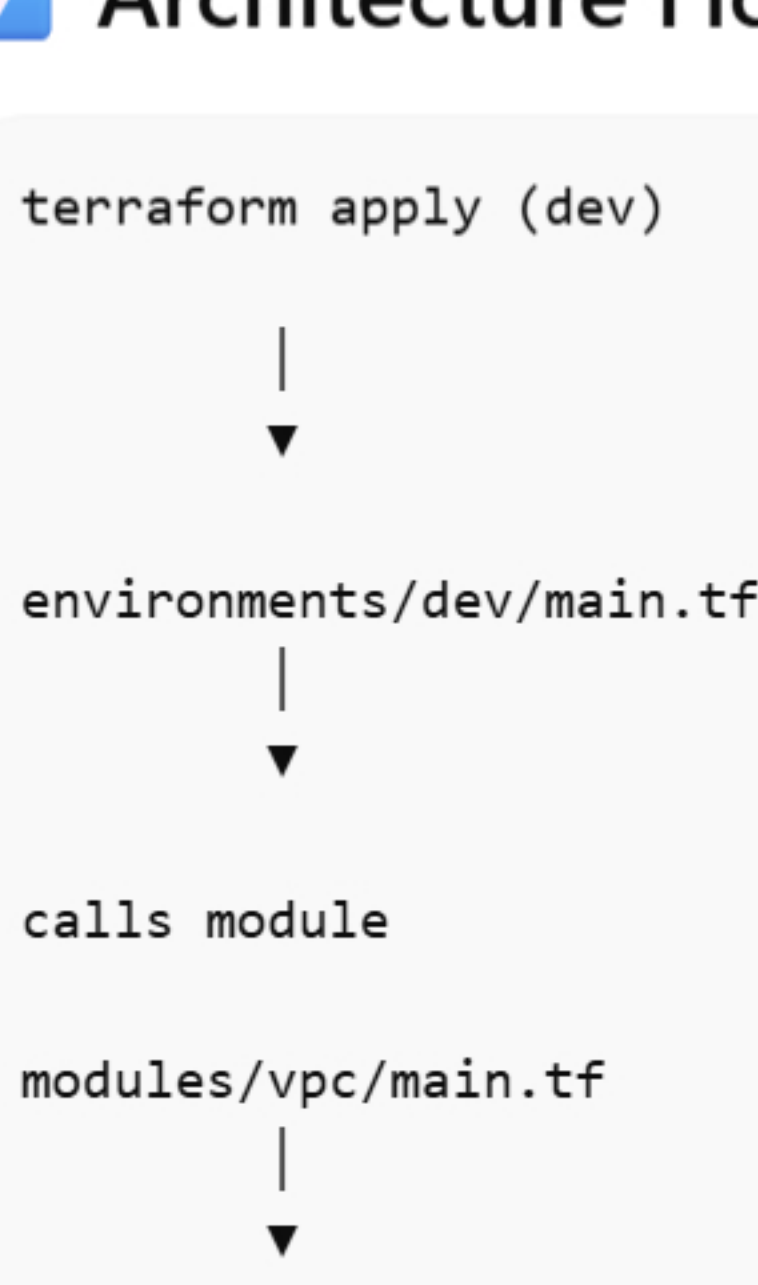
- main.tf
- variables.tf
- outputs.tf

Inject values from

terraform.tfvars

Create **Dev VPC**

7 Architecture Flow



8 Why This Structure Is Used (Real Companies)

Benefits:

- Reusable modules
- Same module used for **dev / stage / prod**
- Only **terraform.tfvars** changes
- Clean separation of environments
- Easy CI/CD pipelines

Example:

```
dev   → 10.0.0.0/16
stage → 10.1.0.0/16
prod  → 10.2.0.0/16
```

But **same VPC module**.

Important Interview Point

Companies design Terraform like this: